

PROPUESTA TÉCNICA: Módulo de Gestión y Generación Dinámica de Contratos

Documento de Discusión Interna del Equipo

Proyecto: Plataforma Cloud (SaaS) para Arriendo Flexible

Elaborado para: Hernán, Erick y Anita

1. Objetivo de la Propuesta

Este documento plantea la arquitectura, flujo y herramientas necesarias para implementar un **Módulo de Contratos Dinámicos** dentro de la plataforma. La meta es permitir que arrendadores y arrendatarios negocien, revisen y aprueben contratos directamente en el sistema antes de enviarlos a firma notarial (FirmaVirtual).

Dado que esto añade complejidad, se propone como un **microservicio desacoplado**, lo que permite desarrollarlo en las etapas finales del proyecto sin poner en riesgo el flujo central de pagos (Mercado Pago) y autenticación.

2. Flujo de Trabajo Secuencial (Máquina de Estados)

Para evitar conflictos de edición bidireccional y proteger legalmente a las partes, el contrato tendrá un ciclo de vida unidireccional y controlado por el backend (Go):

1. **Inyección Inicial (Borrador):** Al confirmar la intención de arriendo, el backend (Go) toma una plantilla HTML predefinida en PostgreSQL (ej. "Contrato Quincho Diario") e inyecta las variables automáticas ({{nombre_arrendador}}, {{RUT}}, {{precio}}, {{fechas}}).
2. **Edición (Rol Arrendador):** A través del frontend (Next.js), el dueño del espacio visualiza el contrato en un editor de texto enriquecido. Puede agregar cláusulas especiales o borrar reglas predefinidas.
3. **Revisión (Rol Arrendatario):** El contrato pasa a estado *En Revisión*. El cliente lo visualiza en Next.js en **modo de solo lectura**.
 - *Si rechaza:* Deja un comentario de retroalimentación (ej. "Por favor, modifica el horario de salida"). El documento vuelve al Arrendador.
 - *Si aprueba:* El contrato pasa al estado *Congelado*.
4. **Congelamiento y Renderizado:** El texto HTML final se bloquea y se envía al Microservicio de Generación, el cual devuelve un archivo PDF inmutable.
5. **Firma Legal:** El backend envía el PDF a FirmaVirtual para su certificación (FEA o ANF) y luego lo resguarda en Google Cloud Storage.

3. Arquitectura y Stack Tecnológico Propuesto

Para no sobrecargar el backend principal en Go, la generación de PDFs se aislará en un

Microservicio Independiente desplegado en su propio contenedor Docker en Google Cloud Run.

3.1 Tecnologías Sugeridas

- **Frontend (Next.js - Interfaz):** Se sugiere utilizar **TipTap** o **Slate.js**. Son librerías *headless* (sin estilos molestos) que permiten integrar un editor tipo "Google Docs" bloqueando campos sensibles (para que el arrendador no pueda editar el RUT del cliente o el precio base).
- **Backend Core (Go - Orquestador):** Se encargará de manejar la base de datos (PostgreSQL), cambiar los estados del contrato y orquestar la comunicación.
- **Microservicio PDF (El Motor de Generación):**
 - *Opción A (Recomendada por tiempo):* **Gotenberg**. Es una imagen de Docker lista para usar. Go le envía el HTML y Gotenberg usa Google Chrome de fondo para devolver un PDF perfecto. Cero código extra.
 - *Opción B (Si queremos programarlo):* **Python (WeasyPrint)**. Python es superior a Go para convertir HTML/CSS a PDF. Si decidimos programar el microservicio nosotros mismos, usaríamos un contenedor ligero con Python (FastAPI + WeasyPrint).
 - *Descarte Técnico:* Se descarta renderizar el PDF nativamente en **Go** porque sus librerías implican dibujar el PDF por coordenadas (X, Y), lo que hace imposible mantener el formato de una plantilla HTML editada.

4. Impacto en la Base de Datos (PostgreSQL)

Se requerirán dos tablas principales para soportar este módulo:

- **contract_templates:**
 - id, tipo_espacio, html_base, created_at.
- **contracts:**
 - id, reserva_id, estado (*DRAFT, REVIEW, REJECTED, LOCKED, SIGNED*), html_actual, pdf_url (Ruta en Cloud Storage), comentarios_rechazo.

5. Estrategia de Implementación y Tiempos

Para garantizar que el proyecto se entregue a tiempo a la profesora, se propone la siguiente estrategia de mitigación de riesgos:

1. **Fase 1 (MVP - Sin este módulo):** El flujo principal se construirá asumiendo un "Contrato Genérico" estático que se genera directamente en PDF y se envía a firmar. Esto asegura la nota aprobatoria.
2. **Fase 2 (Semanas 10-12):** Una vez que la retención de Mercado Pago y el SSO de usuarios funcionen perfectamente, levantaremos el contenedor de **Gotenberg/Python**.
3. **Fase 3:** Anita integra el editor TipTap en Next.js y Hernán/Erick conectan el cambio de estados en Go.

Conclusión para el Equipo:

El aislamiento de esta funcionalidad en un microservicio Docker nos permite "apagarla o

encenderla" sin romper la aplicación. Su implementación elevará considerablemente el nivel de la tesis al demostrar un ecosistema completo (Marketplace + SaaS de Gestión Documental).